

The Ledger: The Managed-Account Workflow

A derivation of §6 (Managed Accounts, Virtual Portfolios, and TRS)
from the primitives, with Addendum A1 (StatesHome)
Convergence of a nine-lens adversarial review

R. Delloye

June 2026 — Workflow note to v10.3

Abstract

A managed account is not a primitive. It is the composition of four that already exist: a wallet partition that holds positions and produces PnL; a mandate unit u_{MA} issued by the manager and held by the client, conserved by the issuance law; a deterministic reset contract that observes performance, crystallises one net cash flow, and resets its baseline; and the three-map state model of Addendum A1, which homes the mandate's terms, the shared observables, and the per-client economic state. This note derives the exact managed-account workflow from those primitives and carries one numerical example through subscription, a trade, a period's NAV, a performance-fee crystallisation against a high-water mark, and redemption, verifying $\sum_w w(u) = 0$ at every step. Nine specialist lenses attacked the workflow with 140 challenge questions; 105 were answered and cleared, and 35 escalated to three framework-level weaknesses that §6 cannot resolve without changing the framework. The escalations are stated plainly in §14; they are the most valuable output of the review, not its defects.

Contents

1 Primitives	2
2 Account Establishment	2
3 Subscription	3
4 Trading under Mandate	3
5 Valuation and NAV	4
6 Fee Logic	4
7 Segregation	5
8 CSA Margin and Collateral	5
9 TRS as Synthetic Managed Account	5
10 Redemption and Wind-down	6
11 Balance-Sheet Substantiation at Account Level	6
12 Worked Example	6

13 Conformance and Flags	7
14 Escalated Weaknesses	8

1 Primitives

Wallet, unit, move, conservation. A wallet $w \in \mathcal{W}$ has state $w_t : \mathcal{U} \rightarrow \mathbb{R}$; $w_t(u)$ is the signed quantity of unit u , negative for a short or obligation. A move $m = (w_s, w_d, u, q, t, \text{source}, \text{meta})$ with $q > 0$ acts by $w_s(u) - = q$; $w_d(u) + = q$. A transaction is a finite set of moves applied atomically. Conservation is algebraic, not a check:

$$Q(u) = \sum_{w \in \mathcal{W}} w_t(u) = 0 \quad \forall u, \forall t. \quad (\text{P1})$$

External counterparties are virtual wallets inside the closed system; the ledger has no “outside” within scope. A contract unit issued by party A to party B obeys the *issuance law* $w_A(u) = -1$, $w_B(u) = +1$, so $\sum_w w(u) = 0$ by the same conservation.

Valuation. $V_t = \sum_u w_t(u) P_t(u)$ with P_t an external price input ($P_t(\text{USD}) = 1$). Conservation guarantees *quantity* integrity only; value moves with P_t . PnL is path-independent: $\text{PnL} = V_{t_1} - V_{t_0}$, and decomposes as $\text{PnL} = \text{PnL}_{\text{price}} + \text{PnL}_{\text{flow}}$. Arithmetic is fixed-precision decimal with bankers’ rounding only at instruction generation, so outputs are bit-identical and replayable.

The three-map state model (Addendum A1). State attaches to three maps: immutable versioned *ProductTerms*[u]; the shared *UnitStatus*[u], a materialised projection of the log — a cached view the log can always rebuild; and per-position *PositionState*[w, u], with an **Option** accessor (**None** “never held” $\neq$ **Some(zero)** “held, now flat”) and a monotone carrier (closed-out rows are retained at zero). *UnitStatus* holds one value per unit, shared — read identically by every holder. Its value changes over time, but *UnitStatus* is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward. There is no wallet-keyed economic-state sector: every per-wallet economic fact is (w, u_{MA}) -keyed. Condition C2 requires each event handler to emit a **StateDelta** that is structurally zero-sum per event class; C3 makes it atomic across all three maps; C4 scopes reads by capability; C11 names the unique handler permitted to mutate each *PositionState* field.

2 Account Establishment

The mandate is a unit. A managed account is opened by issuing the mandate unit u_{MA} from the manager w_M to the client w_C :

```
Move(from: w_M, to: w_C, unit: u_MA, quantity: 1,
     source: "mandate_issue", metadata: "MANDATE_ISSUANCE")
```

giving $w_M(u_{\text{MA}}) = -1$, $w_C(u_{\text{MA}}) = +1$, $\sum_w w(u_{\text{MA}}) = 0$. This is a real two-named-wallet issuance with a real issuer and holder, not the rejected self-reflexive sentinel; conservation holds by the issuance law (P1).

Invariant 2.1 (Mandate is a singleton bilateral contract). $\text{support}(w \mapsto w(u_{\text{MA}})) = \{(w_M, -1), (w_C, +1)\}$ while the mandate is live.

Conservation alone does *not* establish Invariant 2.1: P1 admits $(-5, +5)$ or a three-way split. Identity is pinned by four existing primitives together: Invariant 2.1 for the active phase; the **Option** accessor distinguishing **None** (pre-issuance) from **Some(zero)** (terminated, retained); a *UnitStatus* lifecycle guard admitting issuance only from an unissued status (so a terminated u_{MA} cannot be resurrected); and C8 routing a re-mandate to a fresh u'_{MA} with **SupersededBy**. **What is recorded where.**

Datum	Home
Mandate text, fee schedule, benchmark identity, position limits, HWM/hurdle methodology, crystallisation frequency	<i>ProductTerms</i> [u_{MA}] (immutable, versioned)
Lifecycle stage, superseded_by	<i>UnitStatus</i> [u_{MA}] (shared)
HWM value, entry NAV, accrued fees, mandate-breach flags, subscription/redemption cursor, benchmark NAV at this wallet’s inception	<i>PositionState</i> [w_C, u_{MA}]
Current benchmark level (an externally-sourced value, captured as a logged observation event)	<i>UnitStatus</i> [u_{bench}]

The placement of the per-client *economic* scalars is the subject of escalation E1 (§14): A1 stores them, yet each is a fold of the event log.

Principle 2.2 (Mandate is non-valued). u_{MA} is excluded from the domain of valuation: $P_t(u_{\text{MA}})$ is undefined, not zero. A price on u_{MA} would add $+1 \cdot P_t(u_{\text{MA}})$ to the client’s per-wallet NAV and double-count the underlying exposure; the term cancels only in the global sum.

3 Subscription

Capital enters the managed wallet as a cash move from the client’s external funding wallet w_{ext} (a virtual wallet at the ledger boundary):

```
Move(from: w_ext, to: w_C, unit: USD, quantity: K,
     source: "subscribe", metadata: "SUBSCRIPTION")
```

The move is **tagged** SUBSCRIPTION and indexed by the subscription/redemption cursor in *PositionState*[w_C, u_{MA}]. The tag is load-bearing: it lets the net external flow over any interval be reconstructed as a projection over the stream (§6). At subscription the entry NAV and the initial high-water mark are set to the subscribed capital base; the high-water mark before subscription is **None** (unrepresentable), never 0 — a 0 mark would charge a performance fee on the entire contributed capital at the first crystallisation. The benchmark NAV at inception is recorded from *UnitStatus*[u_{bench}] for performance *reporting*.

4 Trading under Mandate

A trade is an ordinary two-legged transaction against a virtual market wallet w_{mkt} , admitted only if it satisfies the mandate guards.

Definition 4.1 (Mandate guard). *A quantitative mandate constraint is a precondition g on the move-generation function. The contract evaluates g against the current state and the recorded price snapshot; if g fails, no moves are emitted and the state is unchanged.*

Guard determinism is **price-relative**. The claim “same wallet state + same trade $\Rightarrow$ same decision” is false for value-based limits (leverage, concentration-by-value, currency caps): holding positions fixed and doubling a price flips the same trade from admit to reject. Determinism is recovered only when the guard is a pure function of (*ProductTerms*, state, trade, P_t) with P_t an *explicit, snapshot-pinned* input. Two totality obligations follow: a missing price ($P_t(u) = \text{None}$) must make the guard *fail closed* (a typed reject), never a silent admit; and a *passive* breach — a limit crossed by price appreciation with no admitted trade — lies outside the domain of a move-time guard and is detected only by a periodic valuation sweep, not by the guard. The pinned-snapshot requirement is escalation E2.

5 Valuation and NAV

The account NAV is the wallet valuation with the mandate excluded (Principle 2.2):

$$\text{NAV}_t = \sum_{u \neq u_{\text{MA}}} w_{C,t}(u) P_t(u).$$

By state-sufficiency, NAV_t depends only on current balances, current unit state, and current prices — not on history. Performance against the benchmark is computable from the inception benchmark NAV and the current level in $\text{UnitStatus}[u_{\text{bench}}]$; it feeds *reporting*, and feeds the *fee* only when the mandate’s methodology names a benchmark hurdle.

6 Fee Logic

Performance is net of capital flows. Over a reset interval $[t_{k-1}, t_k]$ the performance is the gross NAV change less the net external flow:

$$\text{Perf}_k = (\text{NAV}_{t_k} - \text{NAV}_{t_{k-1}}) - \text{NetFlow}_{[t_{k-1}, t_k]}, \quad \text{NetFlow} = \sum_{\substack{m \text{ tagged sub/redemption} \\ t_{k-1} < m.t \leq t_k}} \pm m.q.$$

The gross change $\text{NAV}_{t_k} - \text{NAV}_{t_{k-1}}$ conflates performance with capital flows: by the PnL decomposition a mid-period subscription is a *flow*, and crystallising it as performance would charge a fee on the client’s own contributed capital. NetFlow is a projection over the already-tagged cursor moves (§3); the correction adds no new coordinate.

Fee is not PnL settlement. A single whole-Perf $\rightarrow$ beneficiary move is the *internal desk* case, where a book is swept to Treasury. A client managed account pays the manager only a fee fraction; the gain *stays in the client book*. The fee handler is total over $\text{sign}(\text{NAV} - \text{HWM})$:

$$\begin{aligned} \text{management fee } f_k^m &= r_m \cdot \Delta t_k \cdot \text{NAV}_k^{\text{base}} && (\text{accrues on AUM regardless of sign}), \\ \text{performance fee } f_k^p &= r_p \cdot \max(0, \text{NAV}_k^{\text{net}} - \text{HWM}_{k-1}) && (\text{floored at zero; no clawback}), \\ \text{HWM}_k &= \max(\text{HWM}_{k-1}, \text{NAV}_k^{\text{net}} - f_k^p) && (\text{monotone ratchet; C11 writer } \text{fee_crystallise}), \end{aligned}$$

where $\text{NAV}_k^{\text{net}} = \text{NAV}_{t_k} - f_k^m$. On a loss period the performance fee is 0 and the high-water mark is unchanged, while the management fee still accrues.

Crystallisation is double-entry. A fee is realised as a real cash move $w_C \rightarrow w_M$, so it is conserved and carries its own contra-entry; a non-conserved stored scalar would be single-entry and sit outside P1. The handler emits, atomically (C3):

```
Move(from: w_C, to: w_M, unit: USD, quantity: f_m_k, metadata: "MGMT_FEE")
Move(from: w_C, to: w_M, unit: USD, quantity: f_p_k, metadata: "PERF_FEE")
```

Theorem 6.1 (Fee zero-sum per crystallisation). $\sum_w \Delta_{\text{fee}}(\text{USD}) = 0$ at each crystallisation: the client pays exactly what the manager receives, creating and destroying no value.

Proof. Each leg is $w_C - = q$; $w_M + = q$; summing over wallets gives $-q + q = 0$ per leg, hence 0 over the transaction. $\square$

Quantity, baseline, residual. The crystallise leg, like every settlement leg in this note, emits $q = |x|$ with direction fixed by $\text{sign}(x)$ and *no* move at $x = 0$; a fixed-direction listing with a signed quantity is non-representable on a loss and illegal at $q = 0$. The reset baseline is the *post-settlement* capital base $B_k = \text{NAV}_{t_k} - (f_k^m + f_k^p) + \text{NetFlow}$; reading the pre-settlement NAV would claw the distributed performance back next period. The rounding residual $\text{Perf}_k - \text{round}(\text{Perf}_k)$ is *retained* in w_C as a conserved un-crystallised remainder, never dropped; it is the reconciling item between economic performance and moved cash. Whether B_k , HWM_k , and the accrued fee are stored or re-derived is escalation E1.

7 Segregation

Theorem 7.1 (Segregation). *No conservative move can perturb a wallet it does not name. Client partitions are mutually isolated iff conservation is combined with capability scoping.*

Conservation alone is not segregation. A move $m = (w_A, w_B, \text{USD}, q)$ between two *different* clients' partitions is perfectly conservative ($\sum_w \Delta = 0$), so $P1$ does not forbid it. The §6 phrasing “conservation enforces segregation by algebra” conflates two independent facts: conservation (CONS, $\sum_w \Delta = 0$) and locality (LOC, $\text{supp}(\Delta) \subseteq \{w_s, w_d\}$). Locality gives “no effect beyond the named wallets”; isolation of *distinct* clients additionally requires an authorisation predicate — C4 capability scoping, forbidding a handler from naming or reading across a (w, u_{MA}) boundary it has no capability for. Segregation is therefore a theorem under $\text{CONS} \wedge \text{LOC} \wedge \text{C4}$, which supplies the logical-segregation component of CASS 6 / MiFID II 16(8). Legal segregation (distinct custodial accounts, trust/nominee) is outside scope: the ledger evidences it, it does not establish it. That C4 is asserted in prose rather than typed on the accessor signature is escalation E4 (§14).

8 CSA Margin and Collateral

The CSA margin contract is a wallet-level smart contract attached to a per-counterparty collateral wallet. It reads the aggregate mark-to-market of the trades the CSA governs *in the real ledger*, computes the required collateral (threshold, minimum transfer amount, eligible-collateral and haircut rules), and emits margin-call moves to or from the collateral wallet. Margin eligibility and haircut are contractual data in *ProductTerms*; the exposure is the observed valuation. For a *synthetic* managed account the netting set is the single real unit u_{TRS} , never the simulated constituents in the virtual ledger: reading those would breach the isolation invariant (§9) and over-margin unmargined positions. The realm tag and C4 realm-scoped reads enforce this. The margin exposure $N_k \cdot \text{TR}_k$ is itself an *observation* of the virtual book lifted across the realm boundary as a number; that this observation is unattested and unversioned is escalation E2.

9 TRS as Synthetic Managed Account

A total return swap connects a virtual ledger $\mathcal{L}_v$ to real counterparty wallets in the real ledger $\mathcal{L}_r$ through periodic cash settlement.

Invariant 9.1 (Ledger isolation). *No move has one endpoint in $\mathcal{L}_v$ and the other in $\mathcal{L}_r$. (Invariant P7.)*

At reset t_k the TRS contract in $\mathcal{L}_r$ *observes* the virtual book's performance and emits one real move of $|\text{Payment}_k|$, $\text{Payment}_k = N_k \cdot \text{TR}_k - N_k \cdot r_k \cdot \Delta t_k$, direction by sign:

```
Move(from: w_payer, to: w_receiver, unit: USD,
     quantity: |Payment_k|, metadata: "TRS_NET_SETTLEMENT")
```

A minimal virtual ledger is not a second engine but a realm $\in \{\text{real}, \text{virtual}\}$ tag enforcing Invariant 9.1. The total return $\text{TR}_k = (V_{t_k}^v - V_{t_{k-1}}^v) / V_{t_{k-1}}^v$ is partial: it requires $V_{t_{k-1}}^v > 0$, an unstated precondition that must be a typed failure, not a division by zero.

Theorem 9.2 (Equivalence to the funded managed account). *The TRS settlement and the funded managed-account fee/PnL settlement are the same ledger operation: one net conservative cash move whose magnitude is a deterministic function of an observed performance and a recorded baseline. The funded book plays the role of $\mathcal{L}_v$; the periodic settlement is the TRS payment.*

Proof sketch. Both reduce to Crystallise(magnitude, from, to) with magnitude = observed performance over the interval and direction by sign; both reset a baseline to the post-settlement value; neither moves a position across a boundary. The settlement *primitive* is one; the *products* differ, and that difference is a CDM (reporting) distinction, not a ledger one: a CDM **TotalReturnSwap** is two-legged ($N \text{ TR}$ and $N r \Delta t$), so the report derives from the retained CDM **BusinessEvent**, never from the netted move (§13). $\square$

Price consistency. $\mathcal{L}_v$ valuation and $\mathcal{L}_r$ settlement must use one price vector P_t . Because Invariant 9.1 forbids a cross-ledger move, a price divergence produces unexplained PnL with *no* internal reconciliation path — the one place the “no internal break” guarantee can be silently voided. The framework asserts consistency in prose but does not bind it; the structural fix (a shared, content-addressed snapshot) is escalation E2.

10 Redemption and Wind-down

Wind-down strikes a final NAV, settles the final fee, returns capital, and retires the mandate. Positions are liquidated against the market wallet; the cash is paid to the client’s external wallet; the mandate unit is returned to the manager. All closed-out rows are retained at zero under the monotone carrier, preserving the audit trail, the final high-water mark, and the quantities needed for tax reconstruction.

```
Move(from: w_C, to: w_mkt, unit: <asset>, quantity: <held>)    -- liquidate
Move(from: w_mkt, to: w_C, unit: USD, quantity: <proceeds>)
Move(from: w_C, to: w_ext, unit: USD, quantity: <final NAV>)  -- capital-out
Move(from: w_C, to: w_M, unit: u_MA, quantity: 1)            -- retire mandate
```

11 Balance-Sheet Substantiation at Account Level

Each account’s balance at any date is a deterministic projection of the move stream filtered to its wallets; client statements, desk PnL, and regulatory position reports differ only in the filter. Within the ledger there is no separate account-level *quantity* record to reconcile against. The move stream substantiates *quantities and provenance*; *valuations* depend on the external price vector, and disclosures and legal status require external evidence (consistent with the dual-valuation section of the main specification). Net settlement is *operational* netting: presenting a balance sheet net requires, in addition, a legally enforceable right of set-off and the intent to settle net (IAS 32.42 / ASC 210-20) — a determination outside the ledger. The accounts’ assets = liabilities + equity reconciliation holds at the quantity level by P1 (the trial balance balances by construction); the valued and classified presentation does not follow from P1 alone.

12 Worked Example

One managed account w_C under mandate u_{MA} issued by manager w_M ; virtual market wallet w_{mkt} ; client external wallet w_{ext} . USD reference. Terms: management fee 2%/yr, performance fee 20% over a high-water mark (no hurdle), quarterly ($\Delta t = 0.25$). Conventions: management fee on end-of-period NAV; performance fee on NAV net of the management fee, above the mark; the mandate is non-valued. Every figure below was verified by a decimal conservation check: $\sum_w w(u) = 0$ holds for *every* unit after *every* move.

Step	Moves (src $\rightarrow$ dst : qty unit)	NAV
A. Mandate issuance	$w_M \rightarrow w_C : 1 u_{MA}$	—
B. Subscription	$w_{ext} \rightarrow w_C : 1,000,000 \text{ USD}$	1,000,000

C. Trade (buy 5,000 AAPL @100)	$w_C \rightarrow w_{\text{mkt}} : 500,000 \text{ USD}; w_{\text{mkt}} \rightarrow w_C : 1,000,000$ 5,000 AAPL	
D. Q1 NAV (AAPL $\rightarrow$ 130)	(price move; no ledger move)	1,150,000
E. Fee crystallisation	$w_C \rightarrow w_M : 5,750 \text{ USD (mgmt)}; w_C \rightarrow w_M : 28,850 \text{ USD (perf)}$	1,115,400
F. Wind-down	$w_C \rightarrow w_{\text{mkt}} : 5,000 \text{ AAPL}; w_{\text{mkt}} \rightarrow w_C : 650,000 \text{ USD}; w_C \rightarrow w_{\text{ext}} : 1,115,400 \text{ USD}; w_C \rightarrow w_M : 1 u_{\text{MA}}$	0

Fee arithmetic (Step E). No capital flow occurred inside the period, so $\text{Perf} = (1,150,000 - 1,000,000) - 0 = 150,000$. Management fee $= 1,150,000 \times 0.02 \times 0.25 = 5,750$. NAV net of management $= 1,144,250$. Performance fee $= 0.20 \times (1,144,250 - 1,000,000) = 28,850$. NAV after both fees $= 1,115,400$. New high-water mark $= \max(1,000,000, 1,115,400) = 1,115,400$; reset baseline $B_1 = 1,115,400$. Fee zero-sum: $\sum_w \Delta_{\text{fee}}(\text{USD}) = -34,600(w_C) + 34,600(w_M) = 0$. **Closing identity (all wallets).** Final non-zero balances: $w_M = +34,600$ (fee revenue), $w_{\text{mkt}} = -150,000$ (the gross gain paid out by the market), $w_{\text{ext}} = +115,400$ (client net). Their sum is $34,600 - 150,000 + 115,400 = 0$. The client received principal 1,000,000 plus net gain 115,400; the gross gain 150,000 equals net 115,400 plus fees 34,600. Per-unit conservation holds across the whole chain: u_{MA} is issued $(+1, -1)$ then returned $(0, 0)$; USD nets to 0 at w_C and globally after every move; AAPL is acquired and liquidated with w_{mkt} as the mirror. Every position row is retained at zero under the monotone carrier.

13 Conformance and Flags

No conformance is asserted; each item is either demonstrated against a cited source or flagged for the responsible owner. The CDM strata named below (**LegalAgreement**, **Trade**, **TotalReturnSwap**, **TradeState**, **BusinessEvent**) are identified at the type level only; pinning them to a named CDM release is owed to the responsible CDM owner and is *flagged*, not asserted here.

- **CDM mandate representation.** u_{MA} maps to the CDM **LegalAgreement** stratum (the ISDA Create-digitised investment-management agreement: parties, fee schedule, benchmark identity, methodology, limits), not the **Trade**/product stratum. A discretionary mandate is a legal agreement, not a tradable contract.
- **CDM TRS representation.** A CDM **TotalReturnSwap** is two-legged; the report is the retained **BusinessEvent** (performance leg and financing leg $N r \Delta t$), never the netted settlement move. The ledger “same mechanism” identity is a settlement-primitive identity, *flagged* as not a CDM/reporting identity.
- **SFTR/EMIR reporting surface.** u_{MA} issuance is *not* reportable: a discretionary mandate is the MiFID II Annex I §A(4) portfolio-management service, not a Section C instrument, not an SFT, not a derivative. This discharges the open risk F5 of Addendum A1. The reportable artefact is never u_{MA} but the underlying book trade or u_{TRS} , each carrying its own UTI/UPI/LEI. Reportability is a per-relationship, regime-cited attribute on the *wrapped* unit’s *ProductTerms*; it is not inferable from the coarse unit-type key (escalation GOV, §14).
- **IFRS classification.** Economic NAV and economic PnL are derived here; accounting classification of fair-value changes (FVTPL/FVOCI/amortised cost) and net presentation (IAS 32.42) are outside scope and require external evidence. Accrued fees, modelled as conserved moves rather than stored scalars, carry their own contra-entry and are auditable as double-entry.

- **CDM TradeState alignment.** That the CDM TradeState-per-Trade view and *PositionState*[*w*, *u*] stay replay-consistent (both projections of one minted event) is asserted, not yet verified; it is the outstanding F6 deliverable (a Rosetta mapping rerun against the three-map schema).

14 Escalated Weaknesses

Of 140 challenge questions, 105 were answered and cleared by every touched lens; 35 escalated to framework-level weaknesses that §6 cannot resolve without changing the framework. They are stated plainly; none was resolved by reshaping a question, narrowing its scope, or assuming away its premise.

E1 — Store-vs-derive economic scalars (an Addendum A1 internal inconsistency). A1 homes the high-water mark, entry NAV, accrued fee, and reset baseline as mutable *PositionState* scalars. Yet each is a *fold* of the event log: the high-water mark is a monotone max over recorded post-crystallisation NAVs, the accrued fee a fold over the rate schedule and the NAV path, the baseline a fold over settlements. A1’s own `first_touch_date` ruling demotes exactly such a value to “not state” because caching it “would create a fold-inconsistency under back-dated corrections.” Storing these scalars therefore re-creates the internal-reconciliation surface the framework claims to abolish: under a back-dated price correction the stored value desyncs from the recomputed fold. *Direction of a fix:* hold these as derived projections, not stored fields; their re-derivation requires recorded prices (E2). A breaking mandate amendment makes this concrete — carrying the high-water mark to the successor unit must be re-derivation from the (corrected) stream, never a copy, on pain of a silent mark reset.

E2 — The observation surface is unattested (the linchpin).

The price vector P_t , the benchmark level, the TRS financing fix r_k , and intraday path observations (barrier touch, realised vol) all strike real, irreversible cash. Each enters the ledger as a logged event — a settlement, a benchmark observation, a rate fix — and *UnitStatus* only caches the latest such value for shared reading; the value is overwritten in place only as a read cache, and every change to it is caused by one of those logged events. *Reproducing the number is therefore assured.* The exact price that struck a fee lives in the immutable settlement event in the log, not only in the overwritten cache cell, so replaying the events rebuilds it exactly: a valued balance sheet is as reproducible as the log itself — as reproducible as the events that produced it, never “only as reproducible as a mutable cell.” Path-independence (P10) holds across replay for the same reason, and because $\mathcal{L}_v$ valuation and $\mathcal{L}_r$ settlement both read the price from the same logged event, a fresh replay reconstructs one price for both. Because the prices are in the log, E1’s economic scalars (high-water mark, baseline, accrued fee) are folds of recorded events, exactly as E1 requires. *What remains open is distinct from reproducibility: attestation.* The settlement payload carries the price as a bare scalar with no attestation envelope — provider key, source, observation timestamp, fallback-chain-as-traversed, signature, snapshot content-address — so the number replays deterministically yet is not *verifiable* to be the genuine settlement level: determinism is not attestation. This attestation gap is recorded separately as the Nazarov finding and is *not* resolved here; it is orthogonal to where *UnitStatus*’s value comes from — the source-of-truth question settled above. The benchmark and r_k moreover get weaker discipline than P_t , with no staleness, fallback, or cross-source check, though the benchmark hurdle is client money. *Direction of a fix (attestation, the Nazarov finding):* the logged price, benchmark, and rate events must carry an attested, content-addressed envelope, with cross-ledger consistency secured by sharing the snapshot identity of the event rather than by after-the-fact reconciliation. It carries two governance sub-items: the oracle-key lifecycle (generation, rotation, revocation — on which the attestation, not the replay, depends)

is unspecified; and every trust assumption at the cash boundary is currently ownerless, which is forbidden for a value-bearing input.

E3 — The closed system is blind to counterparty legal identity.

P1 models an external counterparty as a virtual wallet, correct for *quantity* closure but erasing the counterparty’s legal identity. Dual-sided EMIR/CFTC/SFTR reporting requires two validated ISO-17442 LEIs, a reporting-counterparty determination, FC/NFC($\pm$) or SD/MSP classification, and a paired UTI from the CPMI-IOSCO / EU Art 7 generation waterfall on the real u_{TRS} — none of which is a quantity fact, so none lives in the move stream. The only candidate home, *WalletRegistry*, is declared non-economic and non-versioned; a mutable LEI would let an M&A transfer or lapse retroactively rewrite every historical report on replay, so the binding must be a time-indexed append-only sequence. Moreover a trade-repository *pairing* break is an *external* fact: a closed ledger can produce our side of a report but cannot pair it. *Direction of a fix*: either promote counterparty LEI and classification to first-class, time-versioned, replayable state bound to each externally-facing real wallet, or formally scope LEI as an external reference-data dependency that the minting step joins against and records in the event payload for deterministic replay.

E4 — C4 capability scoping is asserted, not typed.

Segregation (§7) and the no-cross- (w, u_{MA}) -read guarantee depend on C4, but A1’s reference accessor $\text{position}(w, u)$ takes no capability argument, so a handler firing on one mandate can read another’s state. This is answerable by adding a capability parameter to the accessor (a section-level refinement); it is recorded here because it is the typed expression on which Theorem 7.1 rests, and the reference signature does not yet carry it.

E5 — No in-ledger solvency liveness.

Crystallising unrealised mark-to-market can drive the cash wallet negative; since negative is a legal obligation, conservation alone cannot distinguish a *funded* obligation from an *insolvent* overdraft. The ledger faithfully represents the obligation; funding is settlement-layer state, and a client-money wallet carries a non-negativity precondition. The residual — a liveness invariant tying crystallisation to settlement finality — is a boundary property the closed ledger represents but does not guarantee.

Reading of the escalations. They are not failures of the workflow above, which is sound as quantity algebra: every step is expressible as atomic $q > 0$ moves with $\sum_w w(u) = 0$ verified, and the worked example closes to the penny. They are the points at which the *performance and reporting* layer outruns the framework’s current treatment of recorded observations and external identity. E2 is the linchpin: the observation surface already enters as logged events, so replay rebuilds every struck price exactly and E1’s economic scalars are folds of those events; what remains for E2 is *attestation* — an attested, content-addressed envelope that makes the logged numbers verifiable, not merely reproducible (recorded separately as the Nazarov finding). These weaknesses are the agenda for the next framework version.